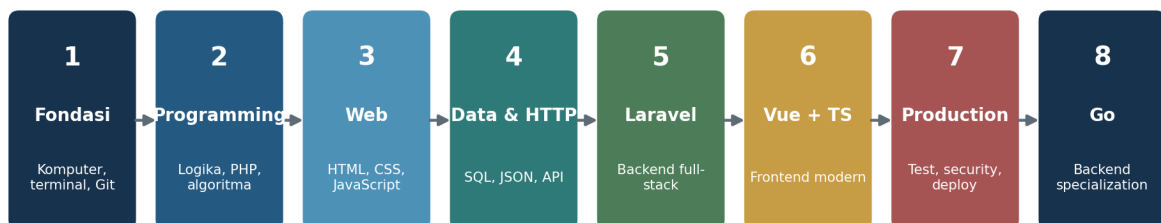


ROADMAP BELAJAR FULL-STACK DEVELOPER 2026

Dari dasar komputer sampai Laravel, Vue, TypeScript, deployment, security, testing, dan Go

Roadmap terpadu: dari nol sampai aplikasi production

Belajar berurutan. Framework datang setelah fundamental.



Prinsip: pahami konsep -> praktik kecil -> proyek -> testing -> deployment -> refleksi

Edisi 25 September 2026

Kurikulum terpadu untuk belajar mandiri dengan proyek nyata

Cara menggunakan buku ini

Buku ini adalah peta belajar sekaligus silabus praktik. Tujuannya bukan membuat kamu menghafal banyak framework, tetapi membangun kemampuan untuk memahami masalah, membaca dokumentasi, menulis kode yang benar, menguji, mengamankan, dan men-deploy aplikasi.

Aturan utama

Jangan naik ke framework berikutnya hanya karena tutorial selesai. Naik level setelah kamu bisa menjelaskan konsep, menulis ulang tanpa menyalin, dan menyelesaikan proyek checkpoint.

Pola belajar setiap topik

Tahap	Apa yang dilakukan	Bukti selesai
1. Pahami	Baca konsep dan mental model	Bisa menjelaskan dengan kata sendiri
2. Ikuti	Tulis contoh kecil	Contoh berjalan dan dipahami
3. Modifikasi	Ubah requirement	Tidak bergantung pada copy-paste
4. Bangun	Kerjakan mini project	Fitur selesai dari nol
5. Uji	Tambahkan test dan kasus gagal	Bug penting tertangkap
6. Refleksi	Catat keputusan dan masalah	Ada catatan belajar / README

Versi teknologi acuan - 25 September 2026

Teknologi	Acuan	Catatan
PHP	8.5.11	Current stable pada 24 Sep 2026
Laravel	13.x	PHP 8.3-8.5; rilis 17 Mar 2026
Vue	Vue 3	Dokumentasi resmi Vue 3
TypeScript	Current stable	Belajar setelah JavaScript
Go	1.27.1	Rilis minor 1 Sep 2026
Database	MySQL atau PostgreSQL	Konsep SQL lebih penting dari vendor
Node.js	Versi LTS yang masih didukung	Digunakan untuk tooling frontend / Vite

Versi dapat berubah. Fokus utama buku ini adalah konsep yang tetap relevan lintas versi.

Daftar isi ringkas

- Bagian I - Fondasi developer dan cara berpikir
- Bagian II - Web fundamental: HTML, CSS, JavaScript, HTTP
- Bagian III - Data dan backend: SQL, PHP, Laravel
- Bagian IV - Frontend modern: Vite, TypeScript, Vue, Inertia
- Bagian V - Quality engineering: testing, security, performance
- Bagian VI - Production: Linux, deployment, Docker, CI/CD
- Bagian VII - Go dan backend specialization
- Bagian VIII - Proyek terpadu, roadmap 26 minggu, checklist, referensi

BAGIAN I - Fondasi developer dan cara berpikir

1. Lingkungan kerja developer

Sebelum framework, kamu perlu nyaman dengan komputer sebagai alat kerja. Banyak kebingungan pemula sebenarnya berasal dari file path, terminal, port, environment variable, atau proses yang tidak dipahami.

Materi wajib

- ☐ File dan folder: absolute path, relative path, extension, hidden file, permission dasar
- ☐ Program, process, service, port, localhost, IP address, DNS dasar
- ☐ Editor: VS Code, workspace, extension secukupnya, integrated terminal
- ☐ Terminal: pwd/cd/ls/mkdir/cp/mv/rm/cat/clear dan ekuivalen Windows
- ☐ Environment variable dan file .env; bedakan konfigurasi dan source code
- ☐ Package manager: Composer untuk PHP dan npm untuk frontend
- ☐ Membaca pesan error: lokasi file, nomor baris, stack trace, error cause

Latihan

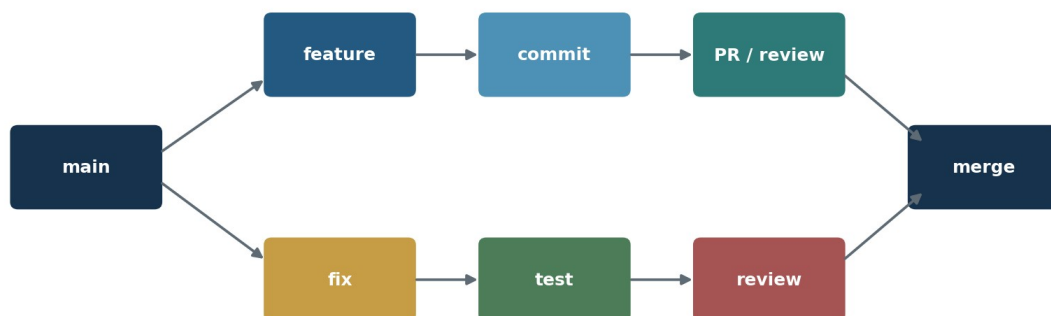
1. Buat struktur folder belajar-programming/ dengan subfolder latihan, notes, dan projects.
2. Dari terminal, buat, pindah, salin, dan hapus file tanpa file explorer.
3. Jalankan server lokal sederhana dan jelaskan arti localhost serta port.

Checkpoint

Kamu lulus tahap ini jika bisa menjelaskan mengapa dua aplikasi tidak bisa memakai port yang sama, apa beda path relatif dan absolut, dan bagaimana proses mencari file .env.

2. Git dan GitHub

Git workflow sederhana yang sehat



Kecilkan perubahan, commit bermakna, test sebelum merge, dan biasakan review.

Git adalah version control. GitHub adalah layanan hosting repository dan kolaborasi. Kuasai Git lokal sebelum bergantung pada tombol di GUI.

Konsep inti

- ☐ Repository, working tree, staging area, commit, HEAD
- ☐ Branch, merge, conflict, remote, clone, fetch, pull, push
- ☐ .gitignore dan alasan secret tidak boleh masuk repository
- ☐ Commit kecil dan pesan commit yang menjelaskan perubahan
- ☐ Feature branch, pull request, review, tag, release

```

git init
git status
git add .
git commit -m "Initial commit"
git switch -c feature/login
git log --oneline --graph
  
```

git diff

Praktik sehat

- Commit setelah satu perubahan logis selesai, bukan setelah satu hari penuh.
- Jangan commit .env, password, token API, private key, atau hasil build yang tidak perlu.
- Sebelum merge: jalankan test, baca diff, dan pastikan tidak ada file rahasia.

3. Fundamental programming dengan PHP

Gunakan PHP sebagai bahasa utama pertama agar konsep backend dan Laravel nanti terasa natural. Tujuan fase ini adalah memahami program tanpa framework.

Urutan konsep

- ☐ Output dan input sederhana
- ☐ Variabel, konstanta, tipe data, casting
- ☐ Operator aritmetika, perbandingan, logika
- ☐ Percabangan if/elseif/else dan match/switch
- ☐ Loop for, while, foreach
- ☐ Function, parameter, return type, scope
- ☐ Array indexed dan associative
- ☐ String manipulation dan date/time dasar
- ☐ Null, truthy/falsy, defensive checks
- ☐ Error, exception, try/catch/finally
- ☐ File I/O sederhana
- ☐ Namespace dan autoloading konsep dasar

```
<?php

function hitungTotal(int $harga, int $jumlah): int
{
    if ($harga < 0 || $jumlah < 0) {
        throw new InvalidArgumentException("Nilai tidak boleh negatif");
    }

    return $harga * $jumlah;
}

echo hitungTotal(15000, 3);
```

Mental model: input -> process -> output

Contoh	Input	Process	Output
Total belanja	harga, jumlah	harga * jumlah	total
Status kelulusan	nilai	bandingkan batas	lulus/tidak
Cari berita	kata kunci	filter data	daftar hasil
Login	email, password	validasi + verifikasi	session / error

4. Algoritma, problem solving, dan struktur data

Kamu tidak perlu memulai dari algoritma kompetitif. Yang dibutuhkan developer aplikasi adalah kemampuan memecah requirement menjadi langkah yang jelas dan memilih struktur data yang tepat.

Materi algoritmik wajib

- ☐ Pseudocode dan flowchart sederhana
- ☐ Decomposition: pecah fitur menjadi submasalah
- ☐ Sequence, selection, iteration
- ☐ Linear search dan binary search konsep
- ☐ Sorting dasar dan kapan menggunakan fungsi bawaan
- ☐ Complexity intuition: $O(1)$, $O(n)$, $O(\log n)$, $O(n^2)$
- ☐ Recursion konsep dan kapan tidak perlu dipakai

Struktur data yang perlu dipahami

- ☐ Array/list
- ☐ Map/dictionary/associative array
- ☐ Set konsep unik
- ☐ Stack dan queue
- ☐ Tree konsep untuk DOM/menu/kategori
- ☐ Graph konsep untuk relasi umum

Yang penting

Jangan menghafal Big-O sebagai teori kosong. Hubungkan dengan pertanyaan: apakah operasi ini bertambah linear ketika data menjadi 10x lebih banyak?

5. OOP dan clean code dasar

- ☐ Class dan object
- ☐ Property dan method
- ☐ Constructor dan dependency
- ☐ Encapsulation
- ☐ Inheritance - gunakan seperlunya
- ☐ Interface dan polymorphism
- ☐ Composition over inheritance
- ☐ Single responsibility
- ☐ Penamaan variabel/fungsi yang jelas
- ☐ Function kecil dengan satu tujuan
- ☐ Hindari duplikasi yang benar-benar sama
- ☐ Refactor setelah test / behavior aman

```
interface Notifier
{
    public function send(string $message): void;
}

final class OrderService
{
    public function __construct(private Notifier $notifier) {}

    public function complete(): void
    {
        // business flow
        $this->notifier->send("Order selesai");
    }
}
```

BAGIAN II - Web fundamental

6. HTML semantik

HTML mendefinisikan struktur dan makna konten. Fokus pada semantic HTML, form, accessibility, dan dokumen yang tetap masuk akal walau CSS dimatikan.

- ☐ doctype, html, head, body, meta viewport, title
- ☐ Heading hierarchy h1-h6
- ☐ header, nav, main, section, article, aside, footer
- ☐ p, strong, em, list, link, image, figure, table
- ☐ Form: label, input, select, textarea, button
- ☐ Input type dan native validation
- ☐ Alt text dan aksesibilitas dasar
- ☐ SEO dasar: title, meta description, canonical konsep
- ☐ Structured content - hindari div untuk semua hal

```
<main>
  <article>
    <h1>Judul Berita</h1>
    <p>Ringkasan berita...</p>
  </article>
</main>
```

Project checkpoint

Buat halaman profil gampong statis: navbar, hero, profil, perangkat, berita, layanan, kontak, footer. Belum memakai JavaScript atau backend.

7. CSS modern dan responsive design

- ☐ Cascade, specificity, inheritance
- ☐ Box model
- ☐ display, position, overflow, z-index
- ☐ Flexbox
- ☐ CSS Grid
- ☐ Responsive design dan mobile-first
- ☐ Media queries
- ☐ Fluid typography dengan clamp()
- ☐ CSS variables / custom properties
- ☐ Pseudo-class :hover, :focus-visible, :disabled
- ☐ Transition dan animation dengan reduced-motion
- ☐ Dark mode dengan prefers-color-scheme atau data-theme
- ☐ Form styling yang tetap accessible

Target UI

Buat layout yang tetap rapi pada layar kecil, zoom 200%, teks panjang, judul dua baris, data kosong, dan mode gelap. Jangan hanya menguji screenshot ideal.

Debug CSS

- ☐ Gunakan DevTools computed style
- ☐ Periksa parent constraint sebelum child
- ☐ Cari fixed width yang menyebabkan overflow
- ☐ Uji min-width: 0 pada flex/grid child
- ☐ Uji panjang teks ekstrem
- ☐ Uji keyboard focus dan contrast

8. JavaScript fundamental

Pelajari JavaScript sebelum Vue dan TypeScript. TypeScript tidak menggantikan pemahaman runtime JavaScript.

- ☐ let/const, primitive, object, array
- ☐ Function declaration, expression, arrow function
- ☐ Scope dan closure konsep
- ☐ Destructuring, spread, rest
- ☐ Array methods: map, filter, find, some, every, reduce
- ☐ Object manipulation
- ☐ Modules: import/export
- ☐ DOM query dan manipulation
- ☐ Event bubbling dan delegation
- ☐ Form handling dan validation
- ☐ Promise
- ☐ async/await
- ☐ try/catch
- ☐ fetch() dan JSON
- ☐ localStorage/sessionStorage dengan batasan keamanan

```
async function loadPosts() {
  const response = await fetch("/api/posts");

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  return response.json();
}
```

9. Browser, DOM, DevTools, dan debugging

- ☐ DOM tree dan event model
- ☐ Network tab: request, response, status, headers, timing
- ☐ Console dan breakpoint
- ☐ Sources debugger
- ☐ Application tab: cookie, storage, cache
- ☐ Performance tab dasar
- ☐ Responsive device emulation
- ☐ CORS error vs server error
- ☐ Cache browser dan hard reload

10. HTTP, URL, JSON, cookie, dan session

Mental model aplikasi web



Debug dari kiri ke kanan: UI -> network -> route -> logic -> query -> response

- ☐ URL: scheme, host, port, path, query, fragment

- ☐ HTTP request: method, headers, body
- ☐ HTTP response: status, headers, body
- ☐ GET, POST, PUT/PATCH, DELETE
- ☐ Status 2xx, 3xx, 4xx, 5xx
- ☐ Content-Type dan Accept
- ☐ JSON serialization
- ☐ Cookie flags: HttpOnly, Secure, SameSite
- ☐ Session vs token konsep
- ☐ HTTPS dan TLS gambaran besar
- ☐ CORS: origin, preflight konsep
- ☐ Caching: Cache-Control konsep

Status	Arti umum	Contoh
200	Berhasil	GET data
201	Dibuat	POST membuat resource
204	Berhasil tanpa body	Delete/Update tertentu
400	Request salah	Payload rusak
401	Belum terautentikasi	Login dibutuhkan
403	Tidak diizinkan	Role tidak punya akses
404	Tidak ditemukan	Resource tidak ada
422	Validasi gagal	Form invalid
429	Terlalu banyak request	Rate limit
500	Server error	Bug / dependency gagal

11. Accessibility, SEO, performance, dan UX engineering

- ☐ Keyboard navigation
- ☐ Focus state
- ☐ Label form
- ☐ Color contrast
- ☐ Alt text
- ☐ Semantic landmarks
- ☐ ARIA hanya jika HTML native tidak cukup
- ☐ Responsive images dan lazy loading
- ☐ Core Web Vitals konsep
- ☐ Image optimization: WebP/AVIF sesuai dukungan
- ☐ Minify/build assets
- ☐ Avoid layout shift
- ☐ SEO: title, description, headings, canonical, sitemap, robots
- ☐ Open Graph / social metadata
- ☐ Structured data bila relevan
- ☐ Empty state, loading state, error state, success state

BAGIAN III - Data, PHP web, dan Laravel

12. Database relasional dan SQL

- ☐ Table, row, column, schema
- ☐ Data type: integer, decimal, varchar, text, date/time, boolean
- ☐ Primary key
- ☐ Foreign key
- ☐ Unique constraint
- ☐ NOT NULL dan default
- ☐ Index dan trade-off write/read
- ☐ CRUD SQL
- ☐ WHERE, ORDER BY, LIMIT
- ☐ Aggregate: COUNT, SUM, AVG
- ☐ GROUP BY, HAVING
- ☐ JOIN: INNER, LEFT
- ☐ Subquery konsep
- ☐ Transaction: begin/commit/rollback
- ☐ ACID intuition
- ☐ Normalization 1NF-3NF konsep
- ☐ N+1 query sebagai masalah aplikasi

```
CREATE TABLE posts (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  title VARCHAR(200) NOT NULL,
  body TEXT NOT NULL,
  published_at DATETIME NULL,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Desain data

Mulai dari requirement, identifikasi entitas, atribut, relasi, constraint, lalu buat migration. Jangan membuat tabel hanya karena nama halaman.

13. PHP untuk web tanpa framework

- ☐ \$_GET dan \$_POST sebagai konsep request data
- ☐ Form submission
- ☐ Server-side validation
- ☐ PDO prepared statement
- ☐ Password hashing dengan password_hash/password_verify
- ☐ Session login
- ☐ CSRF konsep
- ☐ File upload: MIME, size, random filename, storage
- ☐ Redirect-after-POST
- ☐ Separation sederhana: request, logic, data, view

Security

Jangan membangun query SQL dengan string gabungan dari input user. Gunakan prepared statement. Jangan simpan password plaintext.

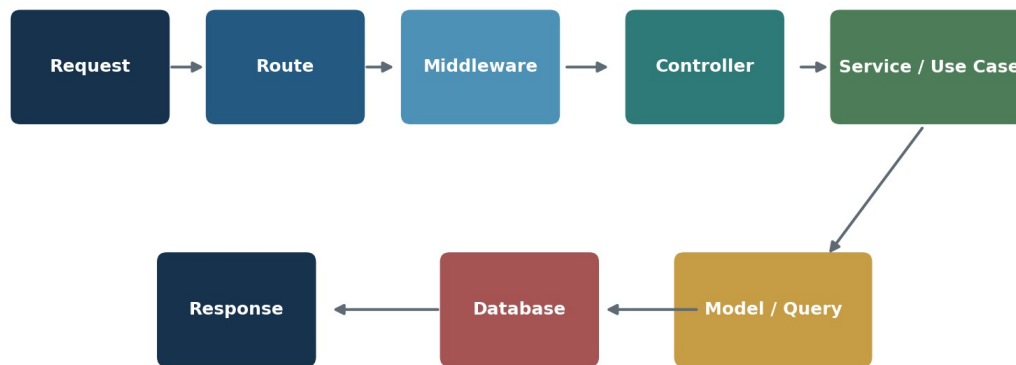
14. Composer, dependency, namespace, autoload

- ☐ composer.json
- ☐ require vs require-dev
- ☐ composer install vs update
- ☐ Semantic versioning konsep

- ☐ PSR-4 autoload
- ☐ Namespace
- ☐ Vendor folder
- ☐ composer.lock dan reproducible install

15. Laravel 13 - fondasi

Laravel request lifecycle - mental model belajar



Tidak semua proyek membutuhkan Service layer. Gunakan struktur sesuai kompleksitas.

Laravel dipelajari setelah kamu memahami HTTP, PHP, SQL, dan OOP dasar. Dengan begitu routing, middleware, Eloquent, dan container tidak terasa seperti sihir.

A. Project structure dan request lifecycle

- ☐ routes/web.php dan routes/api.php sesuai kebutuhan
- ☐ app/Http, app/Models, app/Providers
- ☐ resources/views dan resources/js
- ☐ database/migrations dan seeders
- ☐ storage, config, public
- ☐ artisan commands
- ☐ Environment configuration

B. Routing dan controller

- ☐ Route methods
- ☐ Route parameter
- ☐ Named route
- ☐ Route model binding
- ☐ Resource controller
- ☐ Controller responsibility
- ☐ Form Request untuk validasi kompleks

```
Route::get("/posts/{post}", [PostController::class, "show"])
    ->name("posts.show");
```

C. Blade

- ☐ Layout dan component
- ☐ Escaped output
- ☐ Conditional dan loop
- ☐ Form old input dan validation error
- ☐ Reusable component

- ☐ Avoid business logic berat di template

D. Migration, model, Eloquent

- ☐ Migration up/down
- ☐ Mass assignment
- ☐ Cast
- ☐ Accessor/mutator bila relevan
- ☐ Relationship one-to-one, one-to-many, many-to-many
- ☐ Eager loading
- ☐ Query scopes
- ☐ Pagination
- ☐ Transaction untuk operasi multi-step

E. Validation

- ☐ Request validation
- ☐ Rule umum
- ☐ Custom message
- ☐ Form Request
- ☐ Nested data validation
- ☐ File validation
- ☐ Authorization dalam Form Request bila sesuai

F. Authentication dan authorization

- ☐ Authentication concept
- ☐ Session-based auth
- ☐ Email verification konsep
- ☐ Password reset konsep
- ☐ Middleware auth
- ☐ Gate
- ☐ Policy
- ☐ Role / permission design
- ☐ Least privilege

RBAC

Bedakan authentication (siapa kamu) dari authorization (apa yang boleh kamu lakukan). Role hanya salah satu cara membuat keputusan authorization.

G. Service container dan dependency injection

- ☐ Constructor injection
- ☐ Interface binding bila ada kebutuhan
- ☐ Service provider
- ☐ Avoid service locator pattern berlebihan
- ☐ Testability sebagai alasan dependency injection

H. Storage, upload, mail, notification

- ☐ Filesystem disk
- ☐ Public vs private storage
- ☐ Signed / authorized download
- ☐ Upload validation
- ☐ Mail queue konsep
- ☐ Notification channel konsep

I. Logging, exception, error handling

- ☐ Log level
- ☐ Structured context
- ☐ Exception handling

- ☐ User-safe error message
- ☐ Jangan log password/token/NIK mentah
- ☐ Correlation/request ID konsep

J. Cache, queue, scheduler

- ☐ Cache-aside konsep
- ☐ Cache invalidation
- ☐ Queue job
- ☐ Retry dan idempotency konsep
- ☐ Failed job
- ☐ Scheduler
- ☐ Long-running work jangan blocking request

K. Events dan listeners

- ☐ Event untuk loose coupling
- ☐ Listener
- ☐ Queued listener
- ☐ Jangan menjadikan event sebagai tempat semua logic

L. API Laravel

- ☐ API route
- ☐ JSON resource
- ☐ Pagination
- ☐ Consistent error format
- ☐ Authentication strategy
- ☐ Rate limiting
- ☐ Versioning bila benar-benar dibutuhkan
- ☐ Idempotency untuk operasi tertentu

M. Testing Laravel

- ☐ Feature test
- ☐ Unit test untuk logic murni
- ☐ Database test isolation
- ☐ Factory
- ☐ HTTP assertions
- ☐ Validation/authorization tests
- ☐ File upload test
- ☐ Queue/mail fake bila sesuai

16. Arsitektur aplikasi Laravel yang sehat

Untuk aplikasi kecil, Controller + Model + Form Request + Blade sering sudah cukup. Tambahkan service/use-case class ketika logic bisnis membesar atau digunakan ulang. Hindari pola berlapis hanya untuk terlihat enterprise.

Lapisan	Tanggung jawab utama	Hindari
Route	Pemetaan URL	Business logic
Middleware	Cross-cutting request checks	Query bisnis besar
Controller	Orkestrasi request-response	Ratusan baris logic
Form Request	Validasi input	Query bisnis kompleks
Service / Use Case	Workflow bisnis	Rendering view
Model	Data + relationship	God object
Policy	Authorization resource	UI logic
View	Presentasi	Query dan mutasi data

BAGIAN IV - Frontend modern: Vite, TypeScript, Vue, Inertia

17. Node.js, npm, dan Vite untuk tooling

- ☐ package.json
- ☐ dependency vs devDependency
- ☐ npm install
- ☐ npm run dev
- ☐ npm run build
- ☐ Lockfile
- ☐ ES modules
- ☐ Vite dev server
- ☐ Build output
- ☐ Environment variable frontend dan aturan prefix
- ☐ Jangan expose secret backend ke bundle frontend

18. TypeScript

TypeScript adalah static typechecker untuk JavaScript. Belajar setelah JavaScript agar kamu memahami mana yang dicek sebelum runtime dan mana yang tetap menjadi perilaku JavaScript saat program berjalan.

- ☐ Primitive type
- ☐ Array dan tuple seperlunya
- ☐ Object type
- ☐ type alias
- ☐ interface
- ☐ Union
- ☐ Literal type
- ☐ Narrowing
- ☐ Function parameter dan return type
- ☐ Optional property
- ☐ Generic dasar
- ☐ Utility type
- ☐ Module import/export
- ☐ tsconfig konsep
- ☐ strict mode

```
type User = {
  id: number;
  name: string;
  role: "admin" | "user";
};

function canManage(user: User): boolean {
  return user.role === "admin";
}
```

19. Vue 3

- ☐ Single File Component
- ☐ Composition API
- ☐ script setup
- ☐ ref dan reactive
- ☐ computed
- ☐ watch / watchEffect dengan alasan yang jelas
- ☐ v-if, v-show, v-for

- ☐ v-model
- ☐ Props
- ☐ Emits
- ☐ Slots
- ☐ Component lifecycle
- ☐ Composable
- ☐ Provide/inject seperlunya
- ☐ Vue Router bila SPA murni
- ☐ Form state
- ☐ Loading/error/empty state
- ☐ Accessibility dalam component

```
<script setup lang="ts">
import { ref, computed } from "vue"

const count = ref(0)
const doubled = computed(() => count.value * 2)
</script>

<template>
  <button @click="count++">{{ count }} / {{ doubled }}</button>
</template>
```

Kapan membuat component?

- Ketika UI memiliki tanggung jawab sendiri.
- Ketika pattern digunakan berulang dan API component jelas.
- Bukan hanya karena setiap div harus menjadi component.

20. Laravel + Vue + Inertia

Inertia memungkinkan kamu menggunakan routing/controller Laravel dengan page component Vue. Ini cocok untuk aplikasi full-stack yang ingin frontend modern tanpa memisahkan backend API dan frontend menjadi dua deployment sejak awal.

- ☐ Page component
- ☐ Shared props
- ☐ Form helper / request handling
- ☐ Validation error dari server
- ☐ Layout
- ☐ Partial reload / navigation konsep
- ☐ Authorization tetap di server
- ☐ SSR opsional
- ☐ Vite build

Pilihan arsitektur

Untuk belajar: Laravel + Blade dulu, kemudian Laravel + Vue/Inertia. Pisahkan menjadi REST API + SPA hanya jika tujuan belajar atau kebutuhan proyek memang membutuhkannya.

21. State management dan data fetching

- ☐ Local component state sebagai default
- ☐ Lift state seperlunya
- ☐ Composable untuk reusable logic
- ☐ Server state vs UI state
- ☐ Pinia hanya bila kebutuhan global state jelas
- ☐ Caching data dengan strategi yang dipahami
- ☐ Optimistic update hanya bila recovery failure dirancang

BAGIAN V - Quality engineering: testing, security, performance

22. Testing strategy

Tujuan test adalah memberi kepercayaan saat mengubah kode, bukan mengejar angka coverage tanpa makna.

- ☐ Unit test: logic murni
- ☐ Feature/integration test: request sampai data
- ☐ End-to-end test: alur penting dari sudut user
- ☐ Happy path
- ☐ Validation failure
- ☐ Authorization failure
- ☐ Boundary cases
- ☐ Regression test untuk bug yang pernah terjadi
- ☐ Test data dengan factory
- ☐ Deterministic test

Jenis	Contoh	Kapan utama
Unit	Hitung total / rule domain	Logic murni kompleks
Feature	POST /service-request	Aplikasi Laravel
Integration	DB + repository / API external wrapper	Boundary antar komponen
E2E	Login -> submit -> cek status	Critical user journey

23. Web security wajib

- ☐ Threat modeling sederhana: asset, actor, entry point
- ☐ Input validation
- ☐ Output escaping
- ☐ SQL injection prevention
- ☐ XSS
- ☐ CSRF
- ☐ Authentication
- ☐ Authorization
- ☐ Session fixation/hijacking konsep
- ☐ Password hashing
- ☐ Rate limiting
- ☐ Brute-force protection
- ☐ Secure cookie
- ☐ HTTPS
- ☐ CORS configuration
- ☐ File upload hardening
- ☐ Path traversal awareness
- ☐ Mass assignment
- ☐ IDOR / broken access control
- ☐ Secret management
- ☐ Dependency update
- ☐ Security headers konsep
- ☐ Logging tanpa data sensitif
- ☐ Backup protection
- ☐ Principle of least privilege

Security mindset

Validasi di frontend untuk UX; validasi dan authorization tetap wajib di backend. Jangan percaya input, URL parameter, hidden field, atau role yang dikirim browser.

File upload checklist

- ☐ Batas ukuran
- ☐ Allowlist MIME/extension
- ☐ Nama file acak
- ☐ Simpan file sensitif di lokasi non-public
- ☐ Jangan mengeksekusi file upload
- ☐ Download lewat authorization
- ☐ Scan malware jika konteks production memerlukannya

24. Performance

- ☐ Measure before optimize
- ☐ Network waterfall
- ☐ Database slow query
- ☐ Index sesuai query
- ☐ Eager loading
- ☐ Pagination
- ☐ Cache response/data
- ☐ Queue pekerjaan berat
- ☐ Image optimization
- ☐ Asset code splitting
- ☐ Avoid unnecessary JS
- ☐ Compression
- ☐ CDN konsep
- ☐ Load test dasar
- ☐ Profiling konsep

25. Observability dan debugging production

- ☐ Logs dengan level dan context
- ☐ Error tracking konsep
- ☐ Metrics: request count, latency, error rate
- ☐ Health check
- ☐ Uptime monitoring
- ☐ Database monitoring
- ☐ Queue monitoring
- ☐ Alert yang actionable
- ☐ Runbook untuk incident sederhana

BAGIAN VI - Production, Linux, deployment, Docker, CI/CD

26. Linux dan server dasar

- ☐ Filesystem Linux
- ☐ User/group dan permission
- ☐ Process
- ☐ Port
- ☐ systemd/service konsep
- ☐ SSH
- ☐ Environment variable
- ☐ Package update
- ☐ Firewall konsep
- ☐ Log file
- ☐ Disk usage
- ☐ Timezone dan clock synchronization konsep

```
pwd
ls -la
cd /var/www
ps aux
ss -tulpn
df -h
tail -f storage/logs/laravel.log
```

27. DNS, domain, HTTPS, web server

- ☐ DNS A/AAAA/CNAME konsep
- ☐ Domain -> DNS -> server
- ☐ TLS certificate
- ☐ HTTPS redirect
- ☐ Reverse proxy konsep
- ☐ Nginx/Apache virtual host
- ☐ PHP-FPM konsep
- ☐ Static asset caching
- ☐ Upload size limit sinkron dengan app

28. Deployment Laravel

Deployment production - gambaran besar



Tambahkan HTTPS, backup, logs, monitoring, secret management, dan rollback plan.

- ☐ Build/test sebelum deploy
- ☐ Install dependency production
- ☐ Set APP_ENV/APP_DEBUG
- ☐ Generate/apply app key dengan aman
- ☐ Run migration dengan rencana rollback
- ☐ Storage permission
- ☐ php artisan config:cache / route:cache sesuai kebutuhan
- ☐ Queue worker
- ☐ Scheduler
- ☐ Frontend build
- ☐ Health check setelah deploy
- ☐ Backup sebelum perubahan berisiko
- ☐ Rollback plan

Production rule

APP_DEBUG tidak boleh aktif di production. Secret harus berada di environment/secret store, bukan repository.

29. Docker

Belajar Docker setelah paham aplikasi berjalan tanpa Docker. Docker memaketkan runtime dan dependency; Docker bukan pengganti pemahaman Linux, database, network, atau deployment.

- ☐ Image dan container
- ☐ Dockerfile
- ☐ Layer/cache
- ☐ Port mapping
- ☐ Volume
- ☐ Network
- ☐ Environment
- ☐ docker compose
- ☐ Healthcheck konsep
- ☐ Persistent database volume
- ☐ Build multi-stage konsep
- ☐ Jangan bake secret ke image

30. CI/CD

- ☐ Pipeline trigger
- ☐ Install dependency
- ☐ Lint / static analysis
- ☐ Run test
- ☐ Build frontend
- ☐ Artifact
- ☐ Deploy
- ☐ Migration step
- ☐ Post-deploy smoke test
- ☐ Rollback / failed pipeline
- ☐ Protected branch dan required review

```
push / pull request
-> install dependencies
-> lint + static analysis
-> unit + feature tests
-> build assets
-> deploy
-> smoke test
```

31. Backup dan disaster recovery

- ☐ Database backup otomatis
- ☐ File/storage backup
- ☐ Retention policy
- ☐ Encryption
- ☐ Off-site copy
- ☐ Restore test
- ☐ RPO/RTO konsep
- ☐ Dokumentasikan recovery procedure

BAGIAN VII - Go untuk backend specialization

32. Kapan mulai Go

Mulai Go setelah kamu sudah memahami backend melalui PHP/Laravel. Dengan begitu kamu belajar bahasa dan ekosistem baru, bukan sekaligus belajar HTTP, database, authentication, dan arsitektur dari nol.

Fondasi Go

- ☐ go toolchain dan module
- ☐ package
- ☐ variable dan constant
- ☐ function dan multiple return
- ☐ struct
- ☐ method
- ☐ pointer
- ☐ slice
- ☐ map
- ☐ interface
- ☐ error handling
- ☐ defer
- ☐ generic dasar
- ☐ package organization
- ☐ testing

```
package main

import "fmt"

func total(price, qty int) (int, error) {
    if price < 0 || qty < 0 {
        return 0, fmt.Errorf("invalid value")
    }
    return price * qty, nil
}
```

33. Concurrency Go

Jangan memulai Go dari goroutine. Pahami dulu function, struct, interface, error, package, HTTP, dan test.

- ☐ Goroutine
- ☐ Channel
- ☐ Buffered vs unbuffered channel
- ☐ select
- ☐ sync.WaitGroup
- ☐ Mutex
- ☐ Race condition
- ☐ Context cancellation
- ☐ Timeout
- ☐ Worker pattern
- ☐ Avoid goroutine leak

Concurrency

Concurrency bukan berarti setiap fungsi harus dijalankan sebagai goroutine. Gunakan hanya ketika ada pekerjaan yang memang bisa berjalan independen atau menunggu I/O.

34. Backend Go

- ☐ net/http
- ☐ Routing / mux

- ☐ JSON encode/decode
- ☐ Request validation
- ☐ Context
- ☐ Database/sql
- ☐ Transaction
- ☐ Prepared query
- ☐ Migration tool konsep
- ☐ Middleware
- ☐ Authentication/authorization konsep
- ☐ Structured logging
- ☐ Configuration
- ☐ Graceful shutdown
- ☐ HTTP test
- ☐ Benchmark/profiling dasar

35. Laravel vs Go

Konteks	Laravel	Go
CRUD/CMS/admin	Sangat produktif	Bisa, tetapi lebih manual
Business app	Kuat	Kuat
Rapid development	Unggul	Lebih eksplisit
Concurrent network service	Cukup dengan queue/workers	Sangat cocok
CLI/worker kecil	Bisa	Sangat cocok
Belajar awal backend	Lebih cepat bila jalur PHP	Setelah backend fundamental

Tidak perlu memigrasikan Laravel ke Go hanya karena Go lebih cepat pada benchmark tertentu. Pilih berdasarkan requirement, tim, operasi, complexity, dan bottleneck yang nyata.

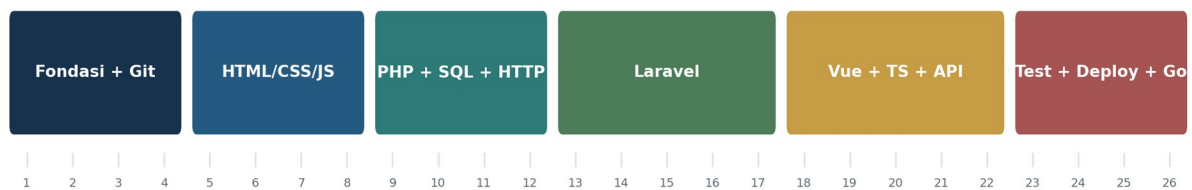
BAGIAN VIII - Proyek terpadu dan roadmap eksekusi

36. Project utama: Portal Gampong bertumbuh 15 level

Level	Target	Materi utama
1	HTML statis	Semantic layout, navigation, section
2	Responsive CSS	Mobile-first, grid/flex, dark mode
3	JavaScript UI	Navbar, modal, search, form state
4	PHP sederhana	Server rendering, form handling
5	MySQL	Schema, CRUD SQL, relationship
6	Authentication	Register, login, session, logout
7	CRUD berita	Create/edit/delete/list/detail
8	Laravel migration	Route, controller, Blade, Eloquent
9	Admin dashboard	CMS, validation, upload
10	RBAC	superadmin/admin/operator/user/guest
11	REST API	JSON resource, rate limit, errors
12	Vue + TypeScript	Component, state, form, API/Inertia
13	Testing	Feature, unit, critical flow
14	Security hardening	CSRF, XSS, upload, authorization, secret
15	Production deploy	HTTPS, backup, queue, logs, CI/CD

37. Roadmap 26 minggu

Contoh roadmap 26 minggu



Kecepatan boleh berubah. Naik level hanya setelah checkpoint kompetensi tercapai.

Minggu	Fokus	Output
1	Environment, terminal, Git	Repo belajar + CLI drills
2	PHP variables, condition, loop	20 latihan logika
3	Function, array, error	CLI mini apps
4	OOP + algoritma dasar	Mini inventory CLI
5	HTML semantic	Landing page
6	CSS responsive	Responsive redesign
7	JavaScript fundamental	Interactive UI
8	DOM, fetch, DevTools	Frontend mini project
9	HTTP + JSON	Request lab
10	SQL CRUD	Database exercises
11	JOIN + schema design	Mini schema gampong
12	PHP web + PDO	Mini blog
13	Laravel routing/controller/Blade	News app
14	Migration/Eloquent/relationship	Data model app
15	Validation/auth/policy	Protected CRUD
16	Upload/mail/logging	Admin module
17	Queue/cache/API/testing	Backend checkpoint
18	Node/Vite + TypeScript	TS exercises
19	Vue component/reactivity	Vue mini app
20	Vue form/composable	Dashboard UI
21	Laravel + Vue/Inertia	Full-stack module
22	API integration/testing	Full-stack checkpoint
23	Security audit	Fix security checklist
24	Linux + deployment	Staging deployment
25	Docker + CI/CD + backup	Automated pipeline
26	Go fundamental + reflection	Go API mini project + portfolio

38. Rutinitas belajar mingguan

Hari	Fokus	Contoh
Senin	Konsep baru	Baca + catat mental model
Selasa	Latihan kecil	5-10 exercise
Rabu	Implementasi	Tambah fitur proyek
Kamis	Debug + test	Cari kasus gagal
Jumat	Refactor	Rapikan struktur dan naming
Sabtu	Project checkpoint	Selesaikan satu vertical slice
Minggu	Review ringan	README, flash notes, rencana minggu depan

39. Sistem checkpoint

- ☐ Saya bisa menjelaskan konsep tanpa membuka catatan
- ☐ Saya bisa membuat contoh kecil dari nol
- ☐ Saya bisa membaca error dan menemukan lokasi masalah
- ☐ Saya bisa mengubah requirement tanpa mengulang tutorial
- ☐ Saya punya test atau langkah verifikasi yang jelas
- ☐ Saya bisa menjelaskan trade-off keputusan saya
- ☐ Kode tersimpan di Git dengan commit yang masuk akal
- ☐ README menjelaskan cara menjalankan project

40. Debugging framework

Ketika bug muncul, jangan mengubah banyak file sekaligus. Persempit masalah secara sistematis.

4. Reproduksi bug dengan langkah konsisten.
5. Baca error message dan stack trace lengkap.
6. Tentukan lapisan: UI, network, route, validation, authorization, business logic, query, storage, deployment.
7. Periksa input aktual dan output aktual.
8. Buat hipotesis kecil dan uji satu perubahan.
9. Tambahkan regression test setelah bug diperbaiki jika memungkinkan.

41. Clean code dan refactoring checklist

- ☐ Nama menggambarkan maksud
- ☐ Function fokus pada satu tugas
- ☐ Controller tidak menjadi god class
- ☐ Validation berada di tempat konsisten
- ☐ Authorization tidak hanya disembunyikan di UI
- ☐ Query tidak diulang tanpa alasan
- ☐ N+1 dicegah
- ☐ Magic number/string penting diberi nama
- ☐ Error message aman untuk user namun cukup detail di log
- ☐ Tidak ada secret / credential di source
- ☐ Dead code dihapus
- ☐ Refactor disertai test atau verifikasi

42. Materi lanjutan setelah roadmap inti

- ☐ Redis lebih dalam
- ☐ WebSocket / realtime
- ☐ Search engine
- ☐ Object storage / CDN
- ☐ Background job architecture
- ☐ Message broker
- ☐ Event-driven architecture
- ☐ OAuth/OIDC
- ☐ Multi-tenancy

- ☐ Database replication konsep
- ☐ System design
- ☐ Load balancing
- ☐ Horizontal scaling
- ☐ Microservices - hanya setelah memahami monolith
- ☐ Cloud platform
- ☐ Infrastructure as Code
- ☐ Advanced observability
- ☐ AI integration / embeddings / agents hanya setelah application fundamentals kuat

43. Yang tidak perlu dipelajari sekaligus

React, Vue, Next, Nuxt, Laravel, Go, Rust, Python, Docker, Kubernetes, dan cloud tidak perlu dipelajari bersamaan. Kedalaman pada satu stack yang selesai sampai production lebih berharga daripada mengenal banyak nama teknologi tanpa mampu menyelesaikan aplikasi.

44. Portfolio yang menunjukkan kemampuan nyata

- ☐ Project live / demo yang dapat dijalankan
- ☐ README: problem, stack, setup, architecture
- ☐ Screenshots seperlunya
- ☐ Database schema atau diagram
- ☐ Testing strategy
- ☐ Security decisions
- ☐ Performance considerations
- ☐ Deployment notes
- ☐ Changelog / release
- ☐ Issue dan commit history yang masuk akal

LAMPIRAN A - Cheat sheet ringkas

Git

```
git status
git switch -c feature/nama
git add .
git commit -m "Add feature"
git fetch
git pull --rebase
git log --oneline --graph --decorate
git diff
```

Composer / Laravel

```
composer install
php artisan about
php artisan route:list
php artisan migrate
php artisan test
php artisan queue:work
php artisan schedule:list
php artisan optimize:clear
```

npm / Vite

```
npm install
npm run dev
npm run build
```

Go

```
go mod init example/app
go fmt ./...
go vet ./...
go test ./...
go run .
go build ./...
```

LAMPIRAN B - Glossary

Istilah	Definisi
API	Kontrak komunikasi antar software.
Authentication	Proses membuktikan identitas.
Authorization	Proses menentukan izin.
Cache	Salinan data untuk mempercepat akses.
CI/CD	Otomasi test/build/deploy.
Component	Bagian UI yang memiliki tanggung jawab dan API.
Controller	Penghubung request ke flow aplikasi.
CRUD	Create, Read, Update, Delete.
Dependency	Kode/library/objek yang dibutuhkan komponen lain.
DOM	Representasi tree dokumen HTML di browser.
Environment variable	Konfigurasi di luar source code.
Framework	Kerangka kerja dengan konvensi dan fasilitas aplikasi.
HTTP	Protokol request-response web.
Idempotent	Operasi berulang menghasilkan efek akhir yang sama.
Migration	Perubahan schema database yang versioned.
ORM	Pemetaan data relasional ke object/model.
Queue	Antrean pekerjaan asynchronous/background.
RBAC	Authorization berdasarkan role.
REST	Gaya desain API berbasis resource dan HTTP.
Session	State server-side untuk interaksi user.
Transaction	Sekelompok operasi database yang commit/rollback bersama.
XSS	Serangan ketika script berbahaya dieksekusi di browser korban.
CSRF	Pemaksaan request menggunakan sesi korban.

SQL injection	Input memengaruhi struktur query SQL secara berbahaya.
---------------	--

LAMPIRAN C - Referensi resmi yang diprioritaskan

Sumber	URL	Gunakan untuk
MDN Learn Web Development	https://developer.mozilla.org/en-US/docs/Learn_web_development	HTML, CSS, JavaScript, accessibility, web fundamentals
PHP Manual	https://www.php.net/manual/	Bahasa PHP dan standard library
Laravel Documentation	https://laravel.com/docs	Laravel 13 dan ekosistem
Vue Guide	https://vuejs.org/guide/	Vue 3
TypeScript Handbook	https://www.typescriptlang.org/docs/handbook/	TypeScript
Go Documentation	https://go.dev/doc/	Go, tutorial, specification, release notes
Git Book	https://git-scm.com/book/en/v2	Git dari dasar sampai workflow
OWASP	https://owasp.org/	Web security awareness
Docker Docs	https://docs.docker.com/	Container fundamentals

Biasakan dokumentasi resmi menjadi sumber pertama. Video dan artikel pihak ketiga berguna untuk perspektif, tetapi dokumentasi resmi lebih baik untuk API, versi, opsi konfigurasi, dan perubahan behavior.

LAMPIRAN D - Checklist kelulusan roadmap inti

- ☐ Bisa menggunakan terminal dan Git tanpa bergantung pada GUI
- ☐ Bisa menulis program PHP kecil dari nol
- ☐ Paham array, function, OOP, error handling, dan problem decomposition
- ☐ Bisa membangun halaman semantic HTML dan responsive CSS
- ☐ Bisa menggunakan JavaScript DOM, event, fetch, async/await
- ☐ Bisa menjelaskan request-response HTTP, cookie, session, status code
- ☐ Bisa mendesain schema SQL dengan key, relation, index, transaction dasar
- ☐ Bisa membuat web PHP + PDO yang aman secara dasar
- ☐ Bisa membuat aplikasi Laravel dengan auth, policy, validation, CRUD, upload, API
- ☐ Bisa menulis feature test untuk alur penting
- ☐ Bisa membangun frontend Vue 3 + TypeScript
- ☐ Bisa menghubungkan Vue dengan Laravel melalui Inertia atau API
- ☐ Bisa melakukan security review dasar
- ☐ Bisa men-deploy aplikasi dengan HTTPS, env, queue, backup, logs
- ☐ Paham Docker dan CI/CD pada level praktis
- ☐ Bisa membuat service Go sederhana dengan HTTP, JSON, DB, test
- ☐ Bisa menjelaskan trade-off monolith vs service terpisah
- ☐ Punya satu project utama yang berkembang dan terdokumentasi

Definisi selesai

Roadmap ini selesai bukan ketika semua kotak dibaca, tetapi ketika kamu dapat membangun, menguji, mengamankan, men-deploy, dan menjelaskan satu aplikasi nyata end-to-end.

Catatan akhir

Urutan yang direkomendasikan tetap: fundamental komputer -> fundamental programming -> HTML/CSS/JavaScript -> HTTP/SQL -> PHP -> Laravel -> Vue/TypeScript -> testing/security/deployment -> Go. Gunakan satu project utama agar setiap materi langsung terhubung ke masalah nyata.

Belajar teknologi adalah proses berulang. Setelah menyelesaikan roadmap, kembali ke area yang terasa lemah, baca dokumentasi lebih dalam, dan refactor proyek berdasarkan pengalaman baru.